Détection d'équipements de protection individuelle en temps réel

Rapport technique — Module Deep Learning Avancé (Axe 3, YOLO)

Omar Ghannem Membre 2
Membre 3
Cycle ingénieur IA & Data Science, 3e année — TEK-UP
Enseignant : Haythem Ghazouani

November 10, 2026

Résumé

Nous présentons un système complet de détection de personnes et d'équipements de protection individuelle (EPI) en milieu de travail, de l'entraînement au déploiement web. Des modèles YOLO sont entraînés sur le dataset public SH17 (8 099 images, 17 classes), suivis avec MLflow, exportés en ONNX et servis par une API FastAPI consommée par une interface React avec webcam. Une règle de conformité déclenche une alerte lorsqu'une tête est détectée sans casque. La chaîne est reproductible (DVC, MLflow, Docker Compose) et testée par GitHub Actions. Le meilleur modèle atteint XX,X % de mAP@50 sur le jeu de test.

1 Problématique et objectifs

Le contrôle du port des EPI est aujourd'hui surtout manuel. Problématique : *comment détecter automatiquement et en temps réel, via une application web, les personnes et leurs équipements de protection, et alerter en cas de non-conformité ?*

Objectifs mesurables : (1) baseline YOLO nano proche des 58,0 % mAP@50 publiés pour YOLOv8n ; (2) comparaison d'au moins deux architectures sur mAP@[.5:.95] et latence ; (3) effet de la résolution d'entrée (640 contre 960) ; (4) export ONNX et gain de latence mesuré ; (5) API et frontend avec webcam et alerte.

2 Données

2.1 Le dataset SH17

SH17 [1] contient 8 099 images issues de Pexels, 75 994 instances et 17 classes, au format YOLO, sous licence CC BY-NC-SA 4.0 (usage académique). L'ordre des identifiants est celui du fichier `sh17.yaml` (alphabétique), différent du README ; nous l'avons vérifié visuellement.

2.2 Exploration et qualité

- Aucune image corrompue, aucune annotation invalide, aucun doublon, aucune fuite train/val.
- Fort déséquilibre : ratio max/min de 118,3 (`hands` 15 850 contre `face-guard` 134) ; les EPI clés sont rares (`helmet` 927, `safety-vest` 530).
- Petits objets : 35,7 % des boîtes font moins de 32 px à 640 px d'entrée (oreilles 92 %, pieds 72 %).
- Densité moyenne de 9,4 objets par image ; résolutions de 1335×1002 à 8192×8192.
- Contenu : surtout des bureaux et ateliers, peu de chantiers.

2.3 Protocole de découpage

Split	Origine	Images
train	90 % du train officiel	5 831
val	10 % du train officiel (seed 42)	648
test	val officiel (comparable au benchmark)	1 620

Le découpage est fait une seule fois, avant toute augmentation, par le stage DVC `prepare` qui réduit aussi les images à 1 280 px de grand côté (les labels normalisés restent valides). Le choix du meilleur modèle se fait sur la validation ; le test ne sert qu'au rapport final.

3 Modélisation

Nous utilisons la famille YOLO (Ultralytics) en transfer learning depuis COCO : un réseau de détection en une passe, adapté au temps réel. Trois runs, même seed et même budget :

1. **baseline** YOLOv8n, 640 px, 30 epochs (même modèle que le benchmark publié) ;
2. **YOLO26n** (ou YOLO11n), 640 px, 30 epochs : architecture récente, sortie sans NMS ;
3. YOLOv8n à **960 px**, 20 epochs : effet de la résolution sur les petits objets.

Augmentations Ultralytics par défaut (mosaïque, HSV, flips, translation/échelle), early stopping (patience 10), sélection sur le mAP@[.5:.95] de validation.

4 Résultats

Run	val mAP50-95	test mAP50	test mAP50-95	AP50 helmet	GPU (ms)
baseline-yolov8n-640					
yolo26n-640					
yolov8n-960					
Référence YOLOv8n [1]	–	58,0	36,6	–	–

Tableau 1: Comparaison des modèles (source : `reports/comparison.csv`, MLflow).

Analyse par taille d'objet. Rappel à IoU 0,5 du meilleur modèle : <16 px : ; 16–32 px : ; 32–96 px : ; >96 px : . Commenter : les petits objets sont-ils la principale source d'erreurs ? Le passage à 960 px aide-t-il ?

Analyse par classe. Commenter les AP de helmet et safety-vest (classes rares) et la matrice de confusion.

5 Export ONNX et latence

Le meilleur modèle est exporté en ONNX (opset 12, entrée fixe). Sur CPU, à entrée identique : PyTorch ms contre ONNX Runtime ms (médiane sur 50 passages), soit un gain de ×. L'écart de mAP@50 entre ONNX et PyTorch sur le test est de point(s) ; s'il est inférieur à 2 points, la version passe de l'alias `staging` à `production` dans le MLflow Model Registry.

6 Architecture logicielle et MLOps

- **DVC** : `dvc.yaml` (stage `prepare`), `params.yaml`, `dvc.lock` (empreintes des données) ; aucune donnée ni aucun poids dans Git.
- **MLflow** : paramètres, courbes par epoch, métriques de test par classe, artefacts (poids, matrice de confusion, analyse par taille), Model Registry avec alias.
- **API FastAPI** : `/health`, `/model/metadata`, `/predict`, `/predict/visualize` ; schémas Pydantic, Swagger sur `/docs`, erreurs 400 et 503, modèle ONNX chargé une seule fois au démarrage, inférence hors de la boucle asynchrone.
- **Frontend React** : glisser-déposer, galerie d'exemples, webcam, canvas de boîtes colorées par classe, bandeau d'alerte, latences client et serveur.
- **Docker Compose** : services `mlflow`, `backend`, `frontend` (images multi-stage) lancés par `docker-compose up -build`.
- **GitHub Actions** : black, flake8, 18 tests pytest, eslint, build du frontend, build Docker.

Règle d'alerte. Une tête est conforme si le centre d'un casque se trouve dans sa boîte agrandie de 50 % vers le haut. On part de `head` (abondante) plutôt que de `person`.

7 Limites et perspectives

- Classes EPI rares (`helmet`, `safety-vest`) : fausses alertes possibles ; piste : sur-échantillonnage ciblé ou ajout d'un dataset de chantiers.
- Budget GPU limité (Kaggle) : 20 à 30 epochs au lieu de plusieurs centaines dans le papier.
- Scènes surtout de bureaux : la démo webcam détecte bien `head`, `face`, `hands`, moins bien les casques sans accessoire réel.
- Perspectives : modèle `s`, résolution 1280, suivi multi-objets, quantification INT8.

Références

[1] H. M. Ahmad, A. Rahimi, *SH17: A Dataset for Human Safety and Personal Protective Equipment Detection in Manufacturing Industry*, arXiv:2407.04590, 2024.

[2] G. Jocher et al., *Ultralytics YOLO*, https://github.com/ultralytics/ultralytics.